МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

ОТЧЁТ

по практической работе № 5

«Основы Django. Структура проекта, маршрутизация и view-функции»

Дисциплина: Технологии разработки приложений на базе фреймворков

Семестр: 5 семестр, 2026–2027 учебный год

Студент: Думлер Виктория

Группа: ЭФБО-10-24

Преподаватель: Ящун Татьяна Викторовна

Проект: Система управления пользовательскими ролями

Год выполнения: 2026

1. Выполненные материалы Яндекс Практикума

Материалы раздела 04 «Основы веб-разработки» выполнены. Материалы ПР1–ПР3 не повторялись, консоль и классы сохранены.

Раздел	Выполненные темы
Раздел 04. Основы веб-разработки	Создание проекта; пути и view-функции; вёрстка для бэкендера (HTML, CSS, Bootstrap)

2. Доработка индивидуального проекта

Название: Система управления пользовательскими ролями.

Django-проект: rolely (Django 5.2), manage.py в корне репозитория.

Приложения: homepage, roles, groups. Отдельное приложение пользователей не создавалось — используется django.contrib.auth.

URL и страницы:

- / — главная, index();
- /roles/ — список ролей;
- /roles/<int:role_id>/ — карточка роли, 404 если нет;
- /groups/ — список групп;
- /groups/<int:group_id>/ — карточка группы, 404 если нет.

View загружают JSON функциями ПР3 и показывают объекты Role и Group. HTML собирает page() с Bootstrap 5.3. ORM и формы не делались.

Репозиторий: https://github.com/VikaD2345/-User-role-management-system

3. Результат работы

Созданы Django-проект и три приложения, настроена маршрутизация двух уровней, реализованы view-функции и навигация. Веб-интерфейс показывает данные предметной области. Консоль main.py, классы, JSON и pytest (22 теста) сохранены. flake8 без замечаний.

4. Вывод

ПР5 перевела проект от консольного меню к веб-интерфейсу: браузер отправляет HTTP-запрос, Django выбирает view, страница строится из объектов ПР3. Основа для шаблонов и Django ORM заложена.

КОНТРОЛЬНЫЕ ВОПРОСЫ

Практическая работа № 5

Думлер Виктория · ЭФБО-10-24 · система управления ролями

Ответы привязаны к коду проекта для демонстрации на защите.

Django и веб-разработка

1. Что такое веб-приложение?

Ответ. Программа, с которой пользователь работает через браузер по HTTP. У меня это Django-сервер и страницы ролей и групп.

2. Что такое фреймворк?

Ответ. Готовая структура и набор инструментов. Django задаёт проект, приложения, URL и view.

3. Что такое Django?

Ответ. Веб-фреймворк на Python. В проекте версия 5.2.

4. Чем фреймворк отличается от библиотеки?

Ответ. Библиотеку вызываем мы. Фреймворк вызывает наш код: запрос приходит в urls.py, затем во view.

5. Что такое Django-проект?

Ответ. Общая конфигурация: пакет rolely, settings.py, корневой urls.py, manage.py.

6. Что такое Django-приложение?

Ответ. Часть системы с своей зоной. У меня homepage, roles, groups.

7. Чем проект отличается от приложения?

Ответ. Проект — настройки всей системы. Приложение — одна предметная область и свои views/urls.

8. Для чего используется manage.py?

Ответ. Команды Django: runserver, migrate, check.

9. Для чего используется settings.py?

Ответ. INSTALLED_APPS, язык ru-RU, часовой пояс Europe/Moscow, путь к role_management.

10. Для чего используется urls.py?

Ответ. Связывает адрес с view. В корне — include приложений.

11. Для чего используется views.py?

Ответ. Обрабатывает HTTP-запрос и возвращает HttpResponse.

12. Что такое development-сервер Django?

Ответ. Локальный сервер python manage.py runserver, адрес http://127.0.0.1:8000/.

HTTP

13. Что такое HTTP?

Ответ. Протокол запроса и ответа между браузером и сервером.

14. Что такое HTTP-запрос?

Ответ. Обращение клиента: метод, URL, заголовки. Например GET /roles/2/.

15. Что такое HTTP-ответ?

Ответ. Ответ сервера: код 200 или 404 и тело HTML.

16. Из каких частей состоит HTTP-запрос?

Ответ. Метод, путь, заголовки, при POST — тело.

17. Из каких частей состоит HTTP-ответ?

Ответ. Код статуса, заголовки, тело (HTML-страница).

18. Что такое URL?

Ответ. Адрес ресурса. /roles/2/ — карточка роли с id=2.

19. Что такое маршрут?

Ответ. Шаблон в path(), который сопоставляется с URL.

20. Что происходит после обращения к URL?

Ответ. rolely/urls.py → urls приложения → view → HttpResponse.

21. Что такое HTTP-метод?

Ответ. Действие запроса: GET читать, POST отправлять данные.

22. Чем GET отличается от POST?

Ответ. GET — просмотр страниц. POST — формы, на ПР5 не нужен.

URL и view

23. Для чего используется path()?

Ответ. Описывает маршрут и view. path("/", views.role_detail, name="role_detail").

24. Что такое конвертер пути?

Ответ. Ограничение типа в URL. int — целое число.

25. Для чего используется ?

Ответ. Передать числовой id во view. У меня role_id и group_id.

26. Что такое view-функция?

Ответ. Функция request → HttpResponse. Например role_detail.

27. Какой параметр получает view Django?

Ответ. Первым всегда request. Затем параметры из URL.

28. Что должна вернуть view-функция?

Ответ. Объект HttpResponse, иногда со status=404.

29. Что такое HttpResponse?

Ответ. HTTP-ответ. page() собирает HTML, его кладут в ответ.

30. Как передать параметр из URL во view?

Ответ. Через /; Django передаёт role_id в функцию.

31. Зачем имена URL-маршрутов?

Ответ. name= нужно для ссылок по имени в следующих работах.

32. Почему маршруты отделяют от логики данных?

Ответ. urls.py только адреса. Поиск роли — find_role_by_id из ПР3.

HTML, CSS и Bootstrap

33. Что такое HTML?

Ответ. Язык разметки страницы: html, head, body, ссылки.

34. Для чего используется HTML-документ?

Ответ. Браузер рисует интерфейс. У меня его собирает page().

35. Из каких частей состоит HTML-документ?

Ответ. DOCTYPE, html lang=ru, head (charset, title, CSS), body.

36. Что такое HTML-элемент?

Ответ. Тег с содержимым: h1, a, ul, div.card.

37. Что такое CSS?

Ответ. Стили. В page() есть блок style для фона навигации.

38. Для чего используются CSS-селекторы?

Ответ. Выбирают элементы. nav.nav задаёт фон меню.

39. Что такое наследование стилей?

Ответ. Свойства родителя переходят к дочерним тегам.

40. Что такое Bootstrap?

Ответ. CSS-фреймворк. Подключаю Bootstrap 5.3 с CDN.

41. Для чего используется Bootstrap?

Ответ. Готовые классы: container, btn, card, badge, list-group.

42. Какие преимущества CSS-фреймворка?

Ответ. Единый вид страниц и адаптивность через viewport.

Индивидуальный проект

43. Какая предметная область реализована?

Ответ. Система управления ролями: User, Role, Permission, Group.

44. Какие страницы реализованы?

Ответ. Главная, список ролей, карточка роли, список групп, карточка группы, страница 404.

45. Какие URL соответствуют страницам?

Ответ. /, /roles/, /roles//, /groups/, /groups//.

46. Какие view-функции реализованы?

Ответ. index, page, roles, role_detail, groups, group_detail.

47. Как запрос проходит от URL до HTML?

Ответ. GET /roles/2/ → include roles.urls → role_detail → load JSON → find_role_by_id → page() → HttpResponse.

48. Что перенесено из консоли ПР3?

Ответ. Классы, storage, find_role_by_id, get_group_members, has_permission. Меню main.py сохранено.

49. Почему ещё нет Django ORM?

Ответ. Методичка ПР5: сначала URL, view и HTML. ORM — дальше.

50. Как изменится проект после БД?

Ответ. Классы станут моделями, JSON заменит база, page() — шаблоны.

51. Какие данные используются для страниц?

Ответ. JSON из role_management/data/, объекты ПР3.

52. Что будет на следующих работах?

Ответ. Шаблоны, формы, ORM, создание и редактирование через веб.

Репозиторий: https://github.com/VikaD2345/-User-role-management-system